

Secure and Reliable .NET Application

Technology Used: .NET 7 Console Application

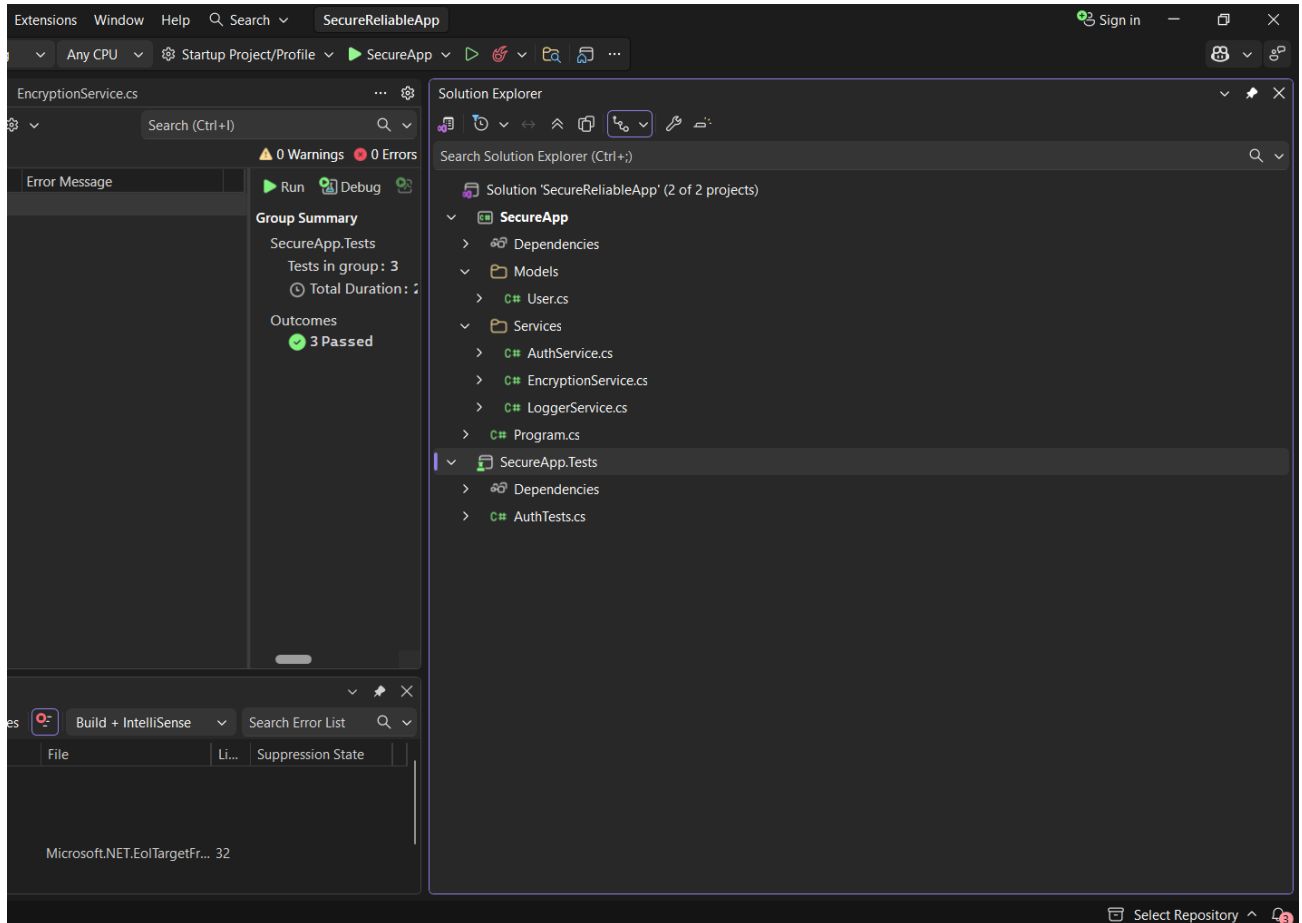
Unit Testing Framework: NUnit

Submitted By: Tushar Dev Singh

Features Implemented:

- Secure User Registration and Login
- Password Hashing using SHA-256
- AES Encryption for Sensitive Data
- Error Handling using try-catch
- Logging to File
- Unit Testing for Authentication and Encryption

Project Structure:



Main Program File

Program.cs:

using SecureApp.Services;

using System;

class Program

{

static void Main()

{

Console.WriteLine("Secure App Running...");

AuthService auth = new AuthService();

EncryptionService enc = new EncryptionService();

auth.Register("tushar", "1234");

bool login = auth.Authenticate("tushar", "1234");

Console.WriteLine("Login success: " + login);

string encrypted = enc.Encrypt("SensitiveData");

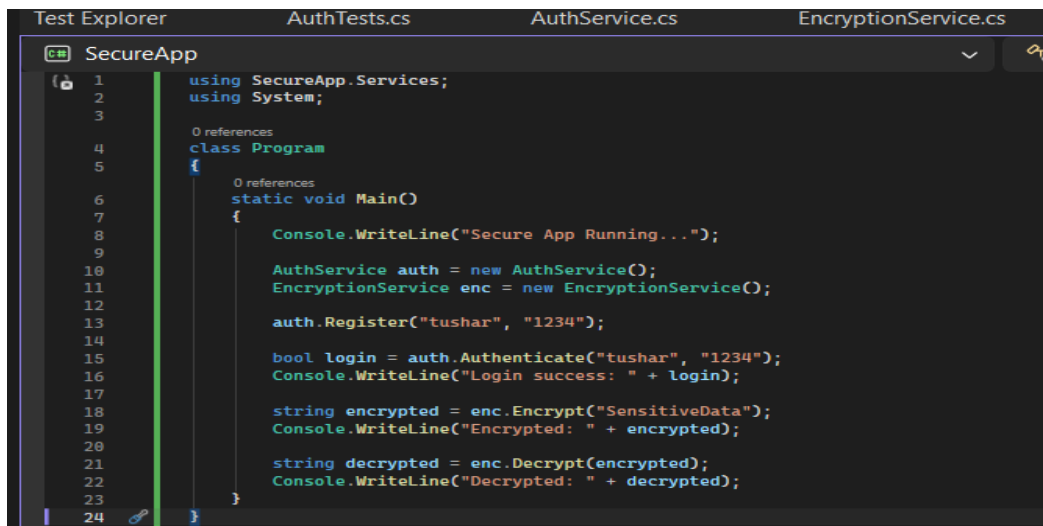
Console.WriteLine("Encrypted: " + encrypted);

string decrypted = enc.Decrypt(encrypted);

Console.WriteLine("Decrypted: " + decrypted);

}

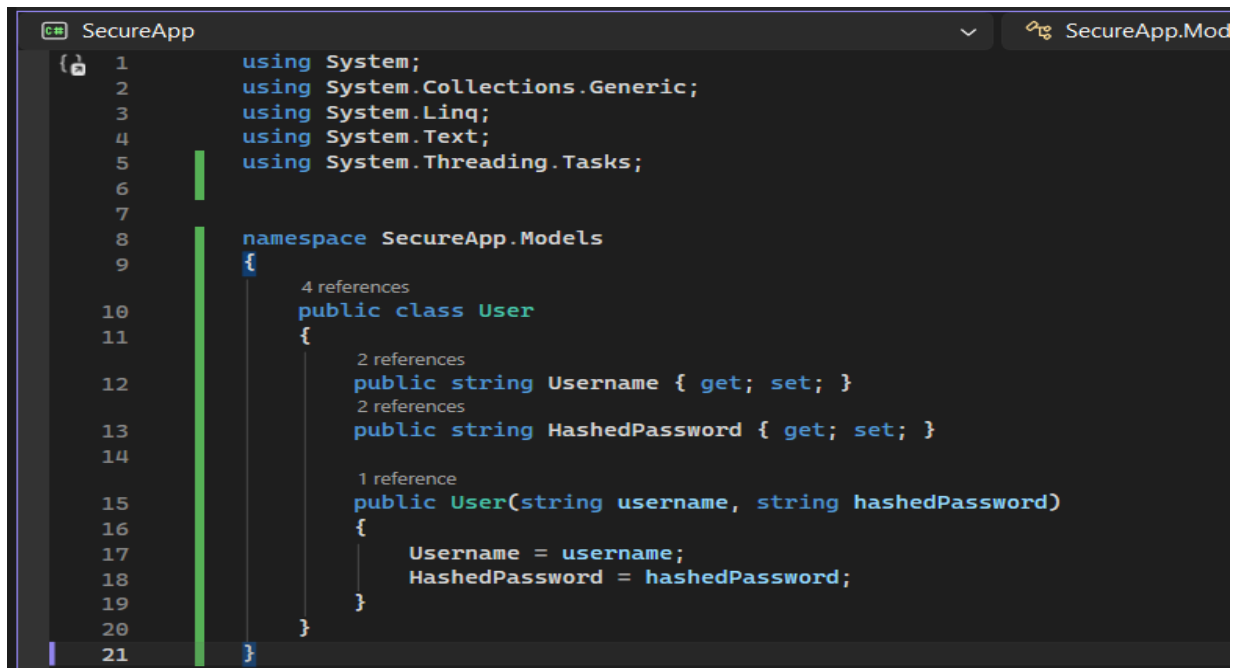
}



User Model Implementation

User.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SecureApp.Models
{
    public class User
    {
        public string Username { get; set; }
        public string HashedPassword { get; set; }
        public User(string username, string hashedPassword)
        {
            Username = username;
            HashedPassword = hashedPassword;
        }
    }
}
```



Authentication Service with SHA256 Hashing

AuthService.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Security.Cryptography;
using SecureApp.Models;
namespace SecureApp.Services
{
    public class AuthService
    {
        private readonly List<User> users = new List<User>();
        private readonly LoggerService logger = new LoggerService();
        public void Register(string username, string password)
        {
            try
            {
                string hash = HashPassword(password);
                users.Add(new User(username, hash));
                logger.Log("User registered: " + username);
            }
            catch (Exception ex)
            {
                logger.LogError(ex.Message);
                throw;
            }
        }
        public bool Authenticate(string username, string password)
        {
            try
            {
                string hash = HashPassword(password);

                foreach (var user in users)
                {
                    if (user.Username == username &&
                        user.HashedPassword == hash)
                    {
                        logger.Log("Login success: " + username);
                        return true;
                    }
                }
            }
        }
    }
}
```

```

        logger.Log("Login failed: " + username);
        return false;
    }
    catch (Exception ex)
    {
        logger.LogError(ex.Message);
        return false;
    }
}

private string HashPassword(string password)
{
    using SHA256 sha = SHA256.Create();
    byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));

    StringBuilder sb = new StringBuilder();
    foreach (var b in bytes)
        sb.Append(b.ToString("x2"));
    return sb.ToString();
}
}
}

```

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System.Security.Cryptography;
7  using SecureApp.Models;
8
9  namespace SecureApp.Services
10 {
11     6 references
12     public class AuthService
13     {
14         private readonly List<User> users = new List<User>();
15         private readonly LoggerService logger = new LoggerService();
16
17         3 references 2/2 passing
18         public void Register(string username, string password)
19         {
20             try
21             {
22                 string hash = HashPassword(password);
23                 users.Add(new User(username, hash));
24                 logger.Log("User registered: " + username);
25             }
26             catch (Exception ex)
27             {
28                 logger.LogError(ex.Message);
29                 throw;
30             }
31         }
32
33         3 references 2/2 passing
34         public bool Authenticate(string username, string password)
35         {
36             try
37             {
38                 string hash = HashPassword(password);
39                 foreach (var user in users)
40                 {
41                     if (user.Username == username &&
42                         user.HashedPassword == hash)
43                     {
44                         logger.Log("Login success: " + username);
45                         return true;
46                     }
47                 }
48                 logger.Log("Login failed: " + username);
49                 return false;
50             }
51             catch (Exception ex)
52             {
53                 logger.LogError(ex.Message);
54                 return false;
55             }
56         }
57
58         2 references
59         private string HashPassword(string password)
60         {
61             using SHA256 sha = SHA256.Create();
62             byte[] bytes = sha.ComputeHash(Encoding.UTF8.GetBytes(password));
63             StringBuilder sb = new StringBuilder();
64             foreach (var b in bytes)
65                 sb.Append(b.ToString("x2"));
66             return sb.ToString();
67         }
68     }
69 }

```

AES Encryption Service

EncryptionService.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO;
using System.Security.Cryptography;
namespace SecureApp.Services
{
    public class EncryptionService
    {
        private readonly byte[] key =
            Encoding.UTF8.GetBytes("12345678901234561234567890123456");
        private readonly byte[] iv =
            Encoding.UTF8.GetBytes("1234567890123456");
        public string Encrypt(string plainText)
        {
            using Aes aes = Aes.Create();
            aes.Key = key;
            aes.IV = iv;
            using MemoryStream ms = new MemoryStream();
            using CryptoStream cs =
                new CryptoStream(ms, aes.CreateEncryptor(), CryptoStreamMode.Write);
            using StreamWriter sw = new StreamWriter(cs);
            sw.Write(plainText);
            sw.Close();
            return Convert.ToBase64String(ms.ToArray());
        }
        public string Decrypt(string cipherText)
        {
            using Aes aes = Aes.Create();
            aes.Key = key;
            aes.IV = iv;
            byte[] buffer = Convert.FromBase64String(cipherText);
            using MemoryStream ms = new MemoryStream(buffer);
            using CryptoStream cs =
                new CryptoStream(ms, aes.CreateDecryptor(), CryptoStreamMode.Read);
            using StreamReader sr = new StreamReader(cs);
            return sr.ReadToEnd();
        }
    }
}
```

SecureApp

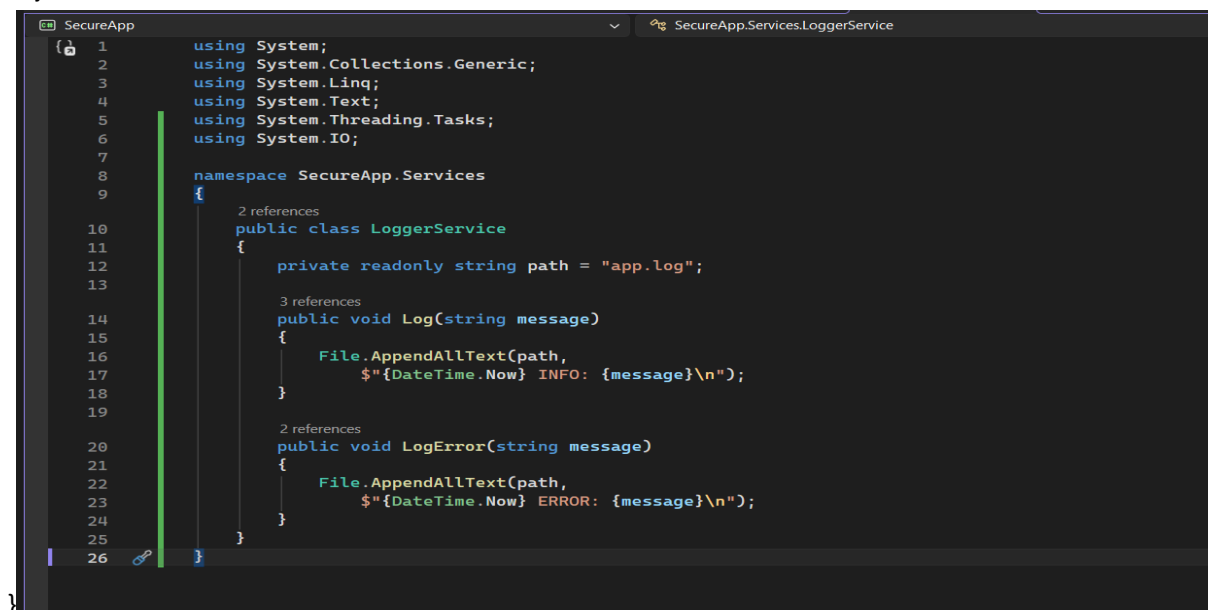
SecureApp.Services.EncryptionService

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6  using System.IO;
7  using System.Security.Cryptography;
8
9
10 namespace SecureApp.Services
11 {
12     4 references
13     public class EncryptionService
14     {
15         private readonly byte[] key =
16             Encoding.UTF8.GetBytes("12345678901234561234567890123456");
17
18         private readonly byte[] iv =
19             Encoding.UTF8.GetBytes("1234567890123456");
20
21         2 references 1/1 passing
22         public string Encrypt(string plainText)
23         {
24             using Aes aes = Aes.Create();
25             aes.Key = key;
26             aes.IV = iv;
27
28             using MemoryStream ms = new MemoryStream();
29             using CryptoStream cs =
30                 new CryptoStream(ms, aes.CreateEncryptor(), CryptoStreamMode.Write);
31             using StreamWriter sw = new StreamWriter(cs);
32
33             sw.Write(plainText);
34             sw.Close();
35
36             return Convert.ToBase64String(ms.ToArray());
37         }
38
39         2 references 1/1 passing
40         public string Decrypt(string cipherText)
41         {
42             using Aes aes = Aes.Create();
43             aes.Key = key;
44             aes.IV = iv;
45
46             byte[] buffer = Convert.FromBase64String(cipherText);
47
48             using MemoryStream ms = new MemoryStream(buffer);
49             using CryptoStream cs =
50                 new CryptoStream(ms, aes.CreateDecryptor(), CryptoStreamMode.Read);
51             using StreamReader sr = new StreamReader(cs);
52
53             return sr.ReadToEnd();
54         }
55     }
56 }
```

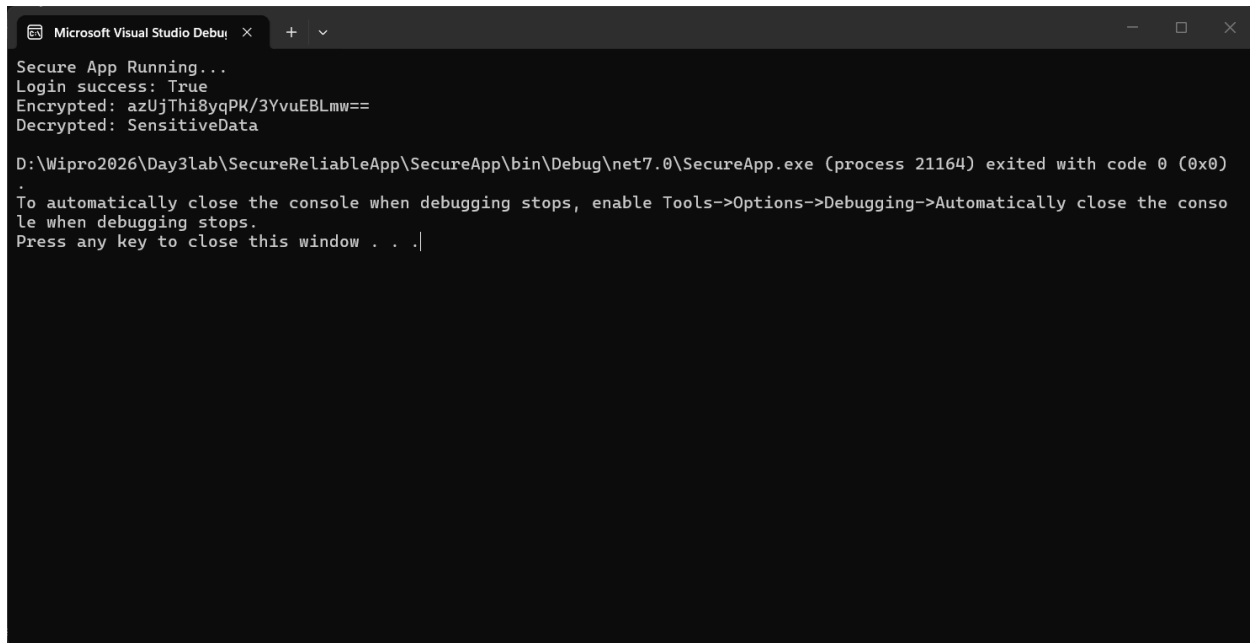
Logging Implementation

LoggerService.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO;
namespace SecureApp.Services
{
    public class LoggerService
    {
        private readonly string path = "app.log";
        public void Log(string message)
        {
            File.AppendAllText(path,
                $"{DateTime.Now} INFO: {message}\n");
        }
        public void LogError(string message)
        {
            File.AppendAllText(path,
                $"{DateTime.Now} ERROR: {message}\n");
        }
    }
}
```



Output



The image shows a screenshot of a Visual Studio Debug Console window. The window has a title bar that says "Microsoft Visual Studio Debug Console" and standard window controls (minimize, maximize, close). The console output is as follows:

```
Secure App Running...
Login success: True
Encrypted: azUjThi8yqPK/3YvuEBLmw==
Decrypted: SensitiveData

D:\Wipro2026\Day3lab\SecureReliableApp\SecureApp\bin\Debug\net7.0\SecureApp.exe (process 21164) exited with code 0 (0x0)
.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Unit Testing using NUnit

AuthTests.cs:

```
using NUnit.Framework;
using SecureApp.Services;

namespace SecureApp.Tests
{
    public class AuthTests
    {
        [Test]
        public void Register_Login_Success()
        {
            AuthService auth = new AuthService();
            auth.Register("user", "123");

            bool result = auth.Authenticate("user", "123");

            Assert.IsTrue(result);
        }

        [Test]
        public void Login_Fails_With_WrongPassword()
        {
            AuthService auth = new AuthService();
            auth.Register("user", "123");

            bool result = auth.Authenticate("user", "wrong");

            Assert.IsFalse(result);
        }

        [Test]
        public void Encryption_Decryption_Works()
        {
            EncryptionService enc = new EncryptionService();

            string encrypted = enc.Encrypt("hello");
            string decrypted = enc.Decrypt(encrypted);

            Assert.AreEqual("hello", decrypted);
        }
    }
}
```

Test ExplorerAuthTests.csAuthService.csEncryptionService.csProgram.csLoggerServ

SecureApp.TestsSecureApp.Tests.AuthTests

```
1 using NUnit.Framework;
2 using SecureApp.Services;
3
4 namespace SecureApp.Tests
5 {
6     0 references
7     public class AuthTests
8     {
9         [Test]
10         0 references
11         public void Register_Login_Success()
12         {
13             AuthService auth = new AuthService();
14             auth.Register("user", "123");
15
16             bool result = auth.Authenticate("user", "123");
17
18             Assert.IsTrue(result);
19         }
20
21         [Test]
22         0 references
23         public void Login_Fails_With_WrongPassword()
24         {
25             AuthService auth = new AuthService();
26             auth.Register("user", "123");
27
28             bool result = auth.Authenticate("user", "wrong");
29
30             Assert.IsFalse(result);
31         }
32
33         [Test]
34         0 references
35         public void Encryption_Decryption_Works()
36         {
37             EncryptionService enc = new EncryptionService();
38
39             string encrypted = enc.Encrypt("hello");
40             string decrypted = enc.Decrypt(encrypted);
41
42             Assert.AreEqual("hello", decrypted);
43         }
44     }
45 }
```

Test Execution Result

The screenshot displays the Visual Studio IDE interface for a project named "SecureReliableApp". The Test Explorer on the left shows a test run summary: "Test run finished: 3 Tests (3 Passed, 0 Failed, 0 Skipped) run in 220 ms". The Test Explorer table lists the test results:

Test	Duration	Traits	Error Message
SecureApp.Tests (3)	22 ms		

The right sidebar shows the "Group Summary" for "SecureApp.Tests":

- SecureApp.Tests
- Tests in group: 3
- Total Duration: 22 ms
- Outcomes: 3 Passed

The Output window at the bottom shows the build logs:

```
Build started at 05:12 PM...
***** Build: 0 succeeded, 0 failed, 2 up-to-date, 0 skipped *****
***** Build completed at 05:12 PM and took 00.142 seconds *****
```

The status bar at the bottom indicates "Test run finished: 3 Tests (3 Passed, 0 Failed, 0 Skipped) run in 220 ms".